# Panoramic Skyline Matching versus Deep Image Retrieval for Geo-Localization in Archipelago Environments

Arvid Norinder and Hjalmar Sandblom




LUND
UNIVERSITY

Department of Mathematics

MSc Thesis TFRT-9999
ISSN 0280–5316

Department of Mathematics
Lund University
Box 118
SE-221 00 LUND
Sweden

© 2025 by Arvid Norinder and Hjalmar Sandblom. All rights reserved.
Printed in Sweden by Media-Tryck.
Lund 2025

# Abstract

# Acknowledgements

# Contents


**1. Introduction** **9**
1.1 Problem formulation . . . . . 10
**2. Theory** **12**
2.1 Skyline as a Geographic Feature . . . . . 12
2.2 Encoding the Skylines . . . . . 12
2.3 Digital Surface Model vs Digital Terrain Model . . . . . 14
2.4 . . . . . 14
2.5 The Domain Gap in Cross-Source Matching . . . . . 14
2.6 Related work . . . . . 15
**3. Methodology** **16**
3.1 Real-World Data Pipeline . . . . . 16
3.2 Simulated DEM Skyline Database Generation . . . . . 17
3.3 Skyline Fingerprinting (Encoding) . . . . . 17
3.4 Localization and Yaw Correction . . . . . 18
3.5 Data . . . . . 19
3.6 Image retrieval . . . . . 19
3.7 Skyline Matching . . . . . 19
3.8 DEM-based approach . . . . . 19
3.9 DEM free approach . . . . . 20
**4. Evaluation** **21**
**5. Discussion and Future Work** **22**
**Bibliography** **23**

# 1

# Introduction

Visual localization is a vital piece in many areas of computer vision and is generally formulated as the problem that inquires, based on visual input such as images and 3D-map, to determine the position in 3D and the orientation, meaning the pitch, yaw and roll, from where a query image is taken, usually referenced as the 6 Degrees of Freedom (6-DoF). Visual localization, as opposed to traditional, signal-based localization with GPS, WiFi or mobile network signals, have two major advantages, they are available everywhere, and secondly they are undisturbable. This makes them interesting in both indoor spaces or densely built urban areas where GPS signals may be unreliable [1]. It is also of great interest for drone, boat and other vehicles that need to operate in GPS-jammed environments.

Visual localization can generally be divided into two methods, the orthodox Direct matching methods and the more novel methods called Hierarchical methods.

Hierarchical methods generally work by a coarse-to-fine 2D-2D-3D paradigm where usually encoded representations of 2D-images are used, together with so called image or global descriptors, that are used to quickly narrow down the potential matches for 2D query image and then for these limited amount of 2D images apply a fine local-descriptor to determine an accurate 6-DoF pose[2].

Visual Place Recognition (VPR) is usually part of the coarse 2D-2D part of this paradigm and aims to answer the fundamentally different question "Do i recognize this place?" instead of "Where am I?". Instead of requiring an exact 6-DoF pose as output, it usually returns the entries in the database (either images or 3D-point clouds) it deems most similar. Now that we have some candidates we can perform fine 2D-3D local matching to get an exact pose. Recent advances in deep learning has made both the coarse and fine candidate retrieval extremely robust and accurate, Neural Networks such as **SuperGlue** are part of the foundation of the popular Hloc toolbox which has become State of the Art (SOTA) for fine retrieval[3][4]. The coarse retrieval have also been revolutionized by models such as AnyLoc, finally culminating in the MegaLoc toolbox that beats or performs on par with other models on most benchmark dataset [5] [6].

For most modern visual localization pipeline, a 3D-point cloud/map of the surroundings has to be created, usually from Structure from Motion (SfM) algorithms such as COLMAP [7][8]. These 3D-maps is built by identifying keypoints in already seen images of the scene and acts as "ground truth" representations of the scene and are used in the fine 2D-3D search to find the the best match.

A big drawback of the visual localization pipeline is that we need to collect images and their pose in order to construct the 3D-map meaning that visual localization might not be suitable in environments where the position of the query image might be "out of bounds" in relation to already collected images. This may also explain why some of the most common benchmark datasets, such as Pittsburgh-30k, Tokyo 24/7, Aachen Day/Night and SF-XL (San Francisco Extra Large), are urban dataset, thanks to their practical implications.

An often overlooked alternative to Visual Localization is Light Detection and Ranging (LiDaR) Place Recognition (LPR) which creates the 3D-point clouds not from already collected images but instead from laser rays. This provides the system with a robust and temporally consistent representation of the scene, not needing training data from both night and day, summer and winter etc [9].

There are a lot of different approaches to solve localization using LiDaR cloud maps. Of particular interest for this work are Skyline Matching. A skyline when visible, defined as the boundary between the terrain and the sky, serves as a powerful feature for geo-localization. It is visible from great distances and its shape is a direct function of the observer's 3D position relative to the surrounding topography. Furthermore, unlike transient features, the macroscopic shape of the terrain skyline is temporally stable, meaning using elevation data as a feature requires only gathering information once that can be reused any season, compared to image data for image retrieval, that has to be gathered over multiple seasons for more consistent matching. By creating a dense database of expected skylines from known locations, a query skyline can be matched against this database to determine the observer's position. Now LPR provides its own set of challenges, primarily the lack of semantic and RGB matching between query and database, something that the descriptors from SfM pipelines easily solves.

## 1.1 Problem formulation

The goal of this thesis is to investigate and evaluate the performance of visual-based geo-localization in archipelago environments. Specifically, this work explores the viability of using 2D skyline representations, derived from both two opposite positioned real-world 200-degree diagonal field of view fisheye camera images, and skylines simulated from digital elevation model (DEM) data, as the primary feature for place recognition. The core challenges addressed include the repetitive and feature-poor nature of archipelago skylines, the roll, pitch and yaw-misalignment between query images and the database, and the 'domain gap' between the real-world data,

which contains artifacts from fisheye undistortion and semantic segmentation, and the simulated data, which has its own inherent inaccuracies stemming from noisy light detection and ranging (LIDAR) point clouds. This thesis aims to develop a robust pipeline that can overcome these challenges to achieve accurate localization, measured by the Euclidean distance between a query's true coordinates and the coordinates of its top-ranked match from the simulated database.

# 2

# Theory

**TODO: segformerb5 model tuning**

## 2.1 Skyline as a Geographic Feature

Because of the high computational load, there is generally a coarse search required before skyline matching [10]. There is also a difference between matching panoramic skyline views and local skyline views, usually including a heavy overhead in data pre-processing for panoramic views[11].

## 2.2 Encoding the Skylines

A database consisting of hundreds of thousands or more of skylines needs to be both reasonable in the amount of memory it requires, but also be efficient in uniquely representing the skylines and quickly return the most similar entry to a queried skyline. To do this we will use a vector-database, where each vector is much smaller in size than the image they represent, and the likeness between two vectors will be determined by their cosine-similarity.

### The Fourier Transform for Signal Representation

The Discrete Fourier Transform (DFT) is a mathematical tool that decomposes a signal into its constituent frequencies. For a 1D signal, such as a skyline vector, the DFT reveals the signal's fundamental frequency components. The magnitude of the Fourier spectrum is of particular interest for pattern matching due to its property of *translation invariance*. If a signal is shifted (i.e., the panoramic skyline is rotated), the magnitude of its Fourier spectrum remains unchanged; only the phase is altered. This makes the Fourier magnitude an ideal candidate for creating a rotationally-invariant fingerprint, which is a key tool for tackling the yaw-misalignment problem when the absolute orientation of a query image is unknown.

### Convolutional Neural Networks and Transfer Learning for Feature Extraction

Convolutional Neural Networks (CNNs) are a class of networks that have revolutionized computer vision by using hierarchical layers of filters to automatically learn features from images. A key technique that leverages the power of pre-trained CNNs is transfer learning.

Transfer learning involves reusing a model pre-trained on a large and diverse dataset (e.g., ImageNet) as a starting point for a new, related task. The rationale is that such models have learned general-purpose visual features (edges, textures, shapes) that are transferable across different domains.

In the context of feature extraction for our skyline encoding, we utilize the pre-trained convolutional base of a CNN. Specifically, the final classification layers of a pre-trained model are removed, and the output from the remaining convolutional layers serves directly as a rich, high-dimensional feature vector, or "fingerprint," encoding the visual content of the input. Crucially, for this application, the pre-trained convolutional base is used in a fixed manner, without any further training or fine-tuning on our specific dataset for feature learning. This approach allows us to efficiently leverage the extensive knowledge gained from training on massive datasets, even when our specific dataset for encoding is comparatively small. In this work, we use MobileNetV2 [**sandler2018mobilenetv2**], a lightweight CNN, as a feature extractor.

### Fine-tuning for Semantic Segmentation

While transfer learning allows us to leverage pre-trained models for feature extraction, complex tasks like semantic segmentation (identifying and classifying regions in an image, such as sky and terrain) often require further adaptation. This is where **fine-tuning** becomes crucial.

Fine-tuning involves taking a pre-trained model and continuing its training on a new, specific dataset for a particular task, often with a different architecture for the final layers. In the context of semantic segmentation for skyline extraction, a pre-trained model (e.g., SegFormer-B5) might have been trained on general segmentation tasks. To optimize its performance for precisely delineating the skyline in our specific real-world imagery, we will employ fine-tuning. This process typically involves:

- **Replacing/Adapting Output Layers:** The original output layers of the pre-trained segmentation model are replaced or adapted to match the number of classes relevant to our skyline segmentation (e.g., sky, terrain).

- **Continued Training:** The entire network, or often just the newly added layers and some of the later pre-trained layers, are trained on our specific dataset of real-world images with ground truth skyline annotations. This allows the model to learn the subtle characteristics of our data and refine its fea-

ture representations to accurately distinguish between sky and terrain, overcoming issues like noise or specific lighting conditions.

- **TODO: Mention SegFormer-B5 specifically and why it's chosen (e.g., efficiency, performance for segmentation).**

This targeted training allows the model to bridge the domain gap between its original training data and our specific application, leading to more accurate and robust skyline segmentation, which is critical for the subsequent geo-localization process.

## 2.3 Digital Surface Model vs Digital Terrain Model

Digital elevation models come in two main categories, digital surface models (DSM), and digital terrain models (DTM). A DSM measures the elevation of the surface, including the elevation of buildings, trees and other structures, while a DTM only measures the elevation of the terrain which structures stand on. Since it is the skyline we are interested in extracting and matching we will be using a DSM model in our work, specifically **TODO: Name of laser data dataset**.

## 2.4

## 2.5 The Domain Gap in Cross-Source Matching

A significant challenge in matching real-world images to simulated data is the domain gap. This problem is two-sided: the data from both sources is imperfect.

- **Real-World Data:** Images are subject to noise from the camera sensor, atmospheric conditions like haze, and artifacts introduced during the fisheye-to-rectilinear undistortion process. Furthermore, the semantic segmentation model that extracts the skyline is not perfect, introducing "fuzzy" or slightly inaccurate boundaries, and sometimes completely missclassifies the ground as sky or the other way around.

- **Simulated Data:** The DSM derived from LiDAR is not a perfect ground truth. It contains its own sources of error, such as sensor noise, interpolation artifacts, and temporal discrepancies (e.g., changes in vegetation height between the time of the LiDAR survey and the time of image capture).

The core matching problem is therefore not to match a "noisy" signal to a "perfect" one, but to find the best correspondence between two different, noisy representations of the same underlying reality.

## 2.6 Related work

### Image Retrieval for Geo-localization

MegaLoc provides a great baseline as it has been shown to be equal or outperform all contemporary models in most environments**TODO: write about what areas megaloc outperforms other models in. VPR only for example?**[5]

### Skyline matching

In [12] Olivier Saurer et.al. simulate a grid of expected skyline representations with the help of a DEM and then compared newly taken images of the mountains with these to perform geo-localization. They used a dictionary of common mountain patterns which they stitched together to gain an encoding for the mountain skyline, later used in numerical comparisons. An advantage they had were the unique skyline features with distinct peaks that the alps contain, although they as well had worse performance and sometimes incorrectly matched skylines due to flat and bland skylines. The archipelago islands which are captured in our image data are not only even more repetitive and bland in their patterns compared to the alps, but are also captured with a dual fisheye camera setup, where the borders of the images are very distorted, causing skyline information loss. Because of these prerequisites, a more complex approach than a preset of common skyline patterns is necessary to encode and identify islands and their current location.

The best method to encode bland skylines is not clear, but fortunately with the emergence of AI-models in the field of computer vision, perhaps the smartest way to encode a skyline is to perform feature extraction from a CNN. In [13], Tang et.al. used a DeepLab CNN to classify which pixels were the skyline, and then they used a CNN in conjunction with simulated skylines from a DEM to perform feature extraction and identify matching skylines, reaching good results. With a $202.6 km^2$ test site in north-eastern Changsha, China, they selected 50 test points to verify the effectiveness of the method, and they achieved an average positioning error of 49.29 meters. Albeit in a different setting with potentially a more feature-rich environment, and the fact that we only have access to their abstract so we do not know how fine coarse their DEM-grid was, their methology in broad strokes is the same as ours.

# 3

# Methodology

This chapter details the complete pipeline developed for skyline-based geo-localization. The methodology is divided into two parallel data preparation pipelines—one for real-world imagery and one for simulated DEM data—which converge at the encoding and matching stages. The entire process is designed to address the orientation misalignments and the domain gap described in the problem formulation.

## 3.1 Real-World Data Pipeline

The goal of this pipeline is to convert raw imagery from a dual-fisheye camera system into standardized, north-centered binary skyline masks. This means that no matter how the boat was oriented when the dual camera images were captured, after rectilinear transforming the images into four 90-degree horizontal field of view (HFOV) images and stitching them together, the north cardinal direction should always be in the center of the real panorama, and the real panoramas should be segmented into sky and non sky. Representing the real images this way will bridge some of the domain gap between the real and simulated panoramas, since the simulated panoramas are also only binary skyline masks built to be rectilinear with the same focal length as the one used to create the four 90-degree HFOV images.

### Image Pre-processing and Orientation Correction

The initial data consists of images from two opposite-facing fisheye lenses.

1. **Roll Correction:** An initial per-image **roll** correction is estimated using a custom-trained CNN. This model is trained on manually annotated binary masks to detect the horizon's tilt within the distorted fisheye view.

2. **Undistortion:** The roll-corrected fisheye images undergo a calibrated undistortion process to convert them into standard rectilinear projections.

3. **Panorama Stitching:** Four resulting rectilinear views are stitched together to form a full 360-degree equirectangular panorama.

4. **Pitch Correction:** A symmetric **pitch** correction is applied during the undistortion process to ensure the horizon is centered vertically in the panoramic frame.

### Skyline Segmentation

Each processed panorama is fed into a Segformer [**xie2021segformer**] semantic segmentation model. The multi-class output is simplified into a binary mask by classifying "sky" pixels as white (255) and all other pixels as black (0). This results in a directory of binary skyline panoramas for each dataset.

## 3.2 Simulated DEM Skyline Database Generation

This pipeline creates the dense, geo-referenced database of simulated skylines used as the ground truth for localization.

### Data Source and its Limitations

The primary data source is a high-resolution LiDAR dataset, processed into a Digital Surface Model (DSM) that captures the elevation of treetops and buildings. It is important to note that this DSM is also a source of noise, containing its own sensor and interpolation inaccuracies, which contributes to the domain gap.

### Database Generation

The generation process is designed to be resilient and memory-safe.

1. **Grid Creation:** A virtual grid of viewpoints is created over the area of interest with a uniform spacing of 25 meters.

2. **Elevation Data Aggregation:** For each viewpoint, a 360-degree skyline is simulated in a massively parallel process. To control memory usage, the process is batched to handle a maximum number of LAZ files concurrently. The raw elevation profiles are saved incrementally to an intermediate database file (‘skyline$_e$levations.npz‘), $making the process fully resumable$.**Mask Generation and Merging:***T*
$safe, with each batch being saved to a separate file. As a final step, these batch files are merged into a si$
$memory ‘zip file‘ - based approach.$

## 3.3 Skyline Fingerprinting (Encoding)

With two sets of masks (real and simulated), the next step is to convert them into compact numerical "fingerprints" for efficient comparison.

### Fourier Transform-based Encoding

The 2D mask is converted to a 1D vector and resampled to a fixed length (4096 points).

3. • **Global Strategy:** The FFT magnitude of the entire vector is computed, resulting in a rotationally-invariant fingerprint ideal for overcoming large **yaw** errors.

   • **Windowed Strategy:** The vector is split into 4 windows, and the FFT magnitudes are computed for each and concatenated, creating a spatially-aware fingerprint.

### CNN-based Encoding

A pre-trained MobileNetV2 is used as a feature extractor. To bridge the domain gap, a ‘transforms.GaussianBlur‘ is applied to both the sharp DEM masks and the softer real masks, making their textural properties more similar before encoding. The panorama is processed in 4 overlapping horizontal windows, and for each window, the feature map is pooled to a $2 \times 2$ grid. This creates a final fingerprint representing a $4 \times 2$ spatial grid, preserving the location of features.

## 3.4 Localization and Yaw Correction

The final step is to match a query fingerprint against the DEM database and calculate the localization error.

1. **Yaw Offset Calculation:** To correct for the final **yaw** misalignment, the real skyline’s 1D vector is iteratively rolled (‘np.roll‘) and compared against the corresponding DEM vector. The pixel shift that minimizes the Mean Squared Error (MSE) between the two vectors is identified as the optimal offset.

2. **Final Similarity Score:** The real fingerprint is rotated by this offset and then matched against the DEM database using a window-averaged cosine similarity metric.

3. **Performance Evaluation:** The top-ranked DEM match is selected. The final performance is quantified by the **localization error**: the Euclidean distance in meters between the query’s known GPS coordinates and the coordinates of its top-ranked DEM match.

## 3.5 Data

## 3.6 Image retrieval

To solve this, we want to leverage both powerful machine learning models, with the current state of the art being MegaLoc, AnyLoc and CliqueMining, skyline matching and a digital elevation model (DEM).

## 3.7 Skyline Matching

## 3.8 DEM-based approach

If accurate DEMs are available we would, just like previous works before, create a database of a grid of some size, where for each cell information is calculated about the expected skyline according to the DEM model. How we encode and store the skyline is not yet clear and needs to be experimented with, but two initial methods come to mind. Either we characterize the skyline of the islands through the height and width of peaks and valleys, and for linear regions calculate the relative width and slope, in an ordered fashion starting clockwise from the north direction, or we simply store the pixeldata for where the skyline is, again in an ordered fashion. In both cases either we train and use a neural network to classify if we have a match or not, or we can think of a way to simply from the numerical representation require a maximum error threshold for when a match has not been found.

For example using the first proposed encoding, if an island, in clockwise order from the north direction, has a concave curve leading to a peak, then a saddle point followed by a longer steeper region before curving down towards the water again, then we could encode it as following, every measurement being in pixels: Find from pixel data the coordinate where the island starts, the peak, the saddle point, where the linear region ends and where the island again meets the water. Store the distance from these coordinates to the coordinates closest to them. For each pair, create a tag describing the relationship between the points. For example the tag for the first two points being compared could be B-P to represent the beginning of the island and peak, then the tag for the next two could be P-S to represent peak and saddle point relation, and so on. These tags should in reality be numerical to gain a completely numerical representation of a view. We then create a string of these tags and view relations that in a 360 view hopefully are unique enough to uniquely describe the location. When receiving a query image (assuming 360-degree fish eye), it is encoded in the same manner and compared to every encoded skyline in the grid, and the best candidate (or some weighted average if this improves performance) is chosen. Perhaps this can be done through the use of a vector database with algorithms like cosine similarity.

## 3.9 DEM free approach

The other path, if an accurate enough DEM is not available, is to create a database only from existing local images of some 3D-scene and use another encoding that this time can leverage that we "see" the coast lines. The first step would be to, together with compass directions, divide the images available into four slices, with their center along the four cardinal directions. We would need to find a method to identify a central point to an island in view, and then in relation to this central point find the angle between the nearest cardinal direction and the viewpoint. The skyline curve and coast line would be extracted from the image and be approximated to be 2D. We would rotate the curves to how we would expect it to look if it were viewed instead from the closest in angle cardinal direction viewpoint. This normalization along the cardinal directions means that we only need to compute one rotation "online" of the new image before it can be compared with the database. Now we can similar to the encoding proposed earlier calculate the height of peaks and valleys, and the slope and width of linear regions, but this time in relation to the shoreline. To avoid having to care about size when identifying islands, we create fractions between the coordinates found and their neighbors. This relation will be invariant to size as long as the resolution of the island is good enough. For each point relation we use the same idea for tags as above.

Each time a new picture is taken, it would be rotated to the nearest cardinal direction, encoded, and then comparing to all encodings in the database for that cardinal direction.

# 4

# Evaluation

# 5

# Discussion and Future Work

# Bibliography


[1] S. T. Nguyen, A. Fontan, M. Milford, and T. Fischer. *Fuseloc: fusing global and local descriptors to disambiguate 2d-3d matching in visual localization*. 2024. arXiv: 2408.12037 [cs.CV]. URL: https://arxiv.org/abs/2408.12037.

[2] P.-E. Sarlin, C. Cadena, R. Siegwart, and M. Dymczyk. "From coarse to fine: robust hierarchical localization at large scale". In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2019, pp. 12716–12725.

[3] P. Sarlin, D. DeTone, T. Malisiewicz, and A. Rabinovich. "Superglue: learning feature matching with graph neural networks". *CoRR* **abs/1911.11763** (2019). arXiv: 1911.11763. URL: http://arxiv.org/abs/1911.11763.

[4] C. Liu, J. Jiao, H. Huang, Z. Ma, D. Kanoulas, and T. Braud. *Air-hloc: adaptive retrieved images selection for efficient visual localisation*. 2025. arXiv: 2403.18281 [cs.CV]. URL: https://arxiv.org/abs/2403.18281.

[5] G. Berton and C. Masone. *Megaloc: one retrieval to place them all*. 2025. arXiv: 2502.17237 [cs.CV]. URL: https://arxiv.org/abs/2502.17237.

[6] N. Keetha, A. Mishra, J. Karhade, K. M. Jatavallabhula, S. Scherer, M. Krishna, and S. Garg. *Anyloc: towards universal visual place recognition*. 2023. arXiv: 2308.00688 [cs.CV]. URL: https://arxiv.org/abs/2308.00688.

[7] J. L. Schönberger and J.-M. Frahm. "Structure-from-motion revisited". In: *Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016.

[8] J. L. Schönberger, E. Zheng, M. Pollefeys, and J.-M. Frahm. "Pixelwise view selection for unstructured multi-view stereo". In: *European Conference on Computer Vision (ECCV)*. 2016.

[9] Y. Zhang, P. Shi, and J. Li. “Lidar-based place recognition for autonomous driving: a survey”. *ACM Computing Surveys* **57**:4 (2024), pp. 1–36. ISSN: 1557-7341. DOI: 10.1145/3707446. URL: http://dx.doi.org/10.1145/3707446.

[10] Z. Pan, J. Tang, T. Tjahjadi, and F. Guo. “Fast geo-location method based on panoramic skyline in hilly area”. *ISPRS International Journal of Geo-Information* **10**:8 (2021). ISSN: 2220-9964. DOI: 10.3390/ijgi10080537. URL: https://www.mdpi.com/2220-9964/10/8/537.

[11] S. Fukuda, S. Nakatani, M. Nishiyama, and Y. Iwai. “Geo-localization using ridgeline features extracted from 360-degree images of sand dunes.” In: *VISIGRAPP (5: VISAPP)*. 2020, pp. 621–627.

[12] O. Saurer, G. Baatz, K. Köser, L. Ladický, and M. Pollefeys. “Image based geo-localization in the alps”. *International Journal of Computer Vision* **116** (2015). DOI: 10.1007/s11263-015-0830-0.

[13] J. Tang, C. Gong, F. Guo, Z. Yang, and Z. Wu. “Geo-localization based on cnn feature matching”. *Optoelectronics Letters* **18**:5 (2022), pp. 300–306. ISSN: 1993-5013. DOI: 10.1007/s11801-022-1148-0. URL: https://doi.org/10.1007/s11801-022-1148-0.